# Trello Project Management Dashboard

Generated by Doxygen 1.13.2

# Chapter 1

# Data Structure Index

## 1.1 Data Structures

Here are the data structures with brief descriptions:

# Chapter 2

# File Index

## 2.1 File List

Here is a list of all files with brief descriptions:

# Chapter 3

# Data Structure Documentation

## 3.1 Project Struct Reference

`#include <project.h>`

### Data Fields

- char * title
- char * description
- char * deadline
- TaskList * task_list
- struct Project * next

### 3.1.1 Detailed Description

Definition at line 7 of file project.h.

### 3.1.2 Field Documentation

#### 3.1.2.1 deadline

`char* deadline`

Definition at line 10 of file project.h.

#### 3.1.2.2 description

`char* description`

Definition at line 9 of file project.h.

#### 3.1.2.3 next

```
struct Project* next
```

Definition at line 12 of file project.h.

#### 3.1.2.4 task_list

```
TaskList* task_list
```

Definition at line 11 of file project.h.

#### 3.1.2.5 title

```
char* title
```

Definition at line 8 of file project.h.

The documentation for this struct was generated from the following file:

- project.h

## 3.2 ProjectList Struct Reference

```
#include <project.h>
```

### Data Fields

- Project * head

### 3.2.1 Detailed Description

Definition at line 15 of file project.h.

### 3.2.2 Field Documentation

#### 3.2.2.1 head

```
Project* head
```

Definition at line 16 of file project.h.

The documentation for this struct was generated from the following file:

- project.h

## 3.3 Task Struct Reference

`#include <task.h>`

### Data Fields

- char * title
- char * description
- char * deadline
- Priority priority_level
- TaskStatus status
- struct Task * next

### 3.3.1 Detailed Description

Definition at line 13 of file task.h.

### 3.3.2 Field Documentation

#### 3.3.2.1 deadline

```
char* deadline
```

Definition at line 17 of file task.h.

#### 3.3.2.2 description

```
char* description
```

Definition at line 16 of file task.h.

#### 3.3.2.3 next

```
struct Task* next
```

Definition at line 20 of file task.h.

#### 3.3.2.4 priority_level

```
Priority priority_level
```

Definition at line 18 of file task.h.

#### 3.3.2.5 status

```
TaskStatus status
```

Definition at line 19 of file task.h.

#### 3.3.2.6 title

```
char* title
```

Definition at line 15 of file task.h.

The documentation for this struct was generated from the following file:

- task.h

## 3.4 TaskList Struct Reference

```
#include <task.h>
```

### Data Fields

- Task * head

### 3.4.1 Detailed Description

Definition at line 23 of file task.h.

### 3.4.2 Field Documentation

#### 3.4.2.1 head

```
Task* head
```

Definition at line 25 of file task.h.

The documentation for this struct was generated from the following file:

- task.h

# Chapter 4

# File Documentation

## 4.1 main.c File Reference

### Functions

- int main ()

### 4.1.1 Function Documentation

#### 4.1.1.1 main()

```
int main ()
```

Definition at line 1 of file main.c.

## 4.2 main.c

Go to the documentation of this file.

```
00001 int main()
00002 {
00003     loading_animation();
00004     start_interface();
00005
00006     return 0;
00007 }
```

## 4.3 priority.c File Reference

```
#include "priority.h"
```

### Functions

- const char * priority_to_string (Priority p)

### 4.3.1 Function Documentation

#### 4.3.1.1 priority_to_string()

```
const char * priority_to_string (
            Priority p)
```

Definition at line 3 of file priority.c.

## 4.4 priority.c

Go to the documentation of this file.

```
00001 #include "priority.h"
00002
00003 const char* priority_to_string(Priority p) {
00004     switch (p) {
00005         case PRIORITY_LOW:
00006             return "Low";
00007         case PRIORITY_MEDIUM:
00008             return "Medium";
00009         case PRIORITY_HIGH:
00010             return "High";
00011         default:
00012             return "Unknown";
00013     }
00014 }
00015
```

## 4.5 priority.h File Reference

**Enumerations**

- enum Priority { PRIORITY_LOW = 1 , PRIORITY_MEDIUM = 2 , PRIORITY_HIGH = 3 }

**Functions**

- const char * priority_to_string (Priority p)

### 4.5.1 Enumeration Type Documentation

#### 4.5.1.1 Priority

```
enum Priority
```

**Enumerator**

| | |
|---|---|
| PRIORITY_LOW | |
| PRIORITY_MEDIUM | |
| PRIORITY_HIGH | |

Definition at line 5 of file priority.h.

### 4.5.2 Function Documentation

#### 4.5.2.1 priority_to_string()

```
const char * priority_to_string (
            Priority p)
```

Definition at line 3 of file priority.c.

## 4.6 priority.h

Go to the documentation of this file.

```
00001 #ifndef PRIORITY_H
00002 #define PRIORITY_H
00003
00004
00005 typedef enum {
00006     PRIORITY_LOW = 1,
00007     PRIORITY_MEDIUM = 2,
00008     PRIORITY_HIGH = 3
00009 } Priority;
00010
00011
00012 const char* priority_to_string(Priority p);
00013
00014 #endif
00015
```

## 4.7 project.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "project.h"
#include "task.h"
#include "ui.h"
```

### Macros

- #define MAX_LINE 256

### Functions

- Project * create_project (const char *title, const char *description, const char *deadline)
- void push_project (ProjectList *list, Project *new_project)
- ProjectList load_project (const char *filename)
- void view_project (ProjectList *list)
- void add_project (ProjectList *list)
- void save_project_to_file (Project *project)
- void free_project_list (ProjectList *list)
- void edit_project (ProjectList *list)
- void delete_project (ProjectList *list)
- void manage_project (ProjectList *list)

### 4.7.1 Macro Definition Documentation

#### 4.7.1.1 MAX_LINE

```
#define MAX_LINE 256
```

Definition at line 9 of file project.c.

### 4.7.2 Function Documentation

#### 4.7.2.1 add_project()

```
void add_project (
            ProjectList * list)
```

Definition at line 138 of file project.c.

#### 4.7.2.2 create_project()

```
Project * create_project (
            const char * title,
            const char * description,
            const char * deadline)
```

Definition at line 12 of file project.c.

#### 4.7.2.3 delete_project()

```
void delete_project (
            ProjectList * list)
```

Definition at line 311 of file project.c.

#### 4.7.2.4 edit_project()

```
void edit_project (
            ProjectList * list)
```

Definition at line 224 of file project.c.

#### 4.7.2.5 free_project_list()

```
void free_project_list (
            ProjectList * list)
```

Definition at line 202 of file project.c.

#### 4.7.2.6 load_project()

```
ProjectList load_project (
            const char * filename)
```

Definition at line 43 of file project.c.

#### 4.7.2.7 manage_project()

```
void manage_project (
            ProjectList * list)
```

Definition at line 386 of file project.c.

#### 4.7.2.8 push_project()

```
void push_project (
            ProjectList * list,
            Project * new_project)
```

Definition at line 28 of file project.c.

#### 4.7.2.9 save_project_to_file()

```
void save_project_to_file (
            Project * project)
```

Definition at line 185 of file project.c.

#### 4.7.2.10 view_project()

```
void view_project (
            ProjectList * list)
```

Definition at line 88 of file project.c.

## 4.8 project.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <string.h>
00004 #include <stdint.h>
00005 #include "project.h"
00006 #include "task.h"
00007 #include "ui.h"
00008
00009 #define MAX_LINE 256
00010
00011
00012 Project* create_project(const char* title, const char* description, const char* deadline)
00013 {
00014     Project* p = (Project*)malloc(sizeof(Project));
00015     if (!p) return NULL;
00016
00017     p->title = strdup(title);
00018     p->description = strdup(description);
00019     p->deadline = strdup(deadline);
00020     p->next = NULL;
00021
00022     p->task_list = (TaskList*)malloc(sizeof(TaskList));
00023     p->task_list->head = NULL;
00024
00025     return p;
00026 }
00027
00028 void push_project(ProjectList* list, Project* new_project)
00029 {
00030     if (!list->head)
00031     {
00032         list->head = new_project;
00033         return;
00034     }
00035
00036     Project* current = list->head;
00037     while (current->next)
00038         current = current->next;
00039
00040     current->next = new_project;
00041 }
00042
00043 ProjectList load_project(const char* filename)
00044 {
00045     ProjectList list;
00046     list.head = NULL;
00047
00048     FILE* file = fopen(filename, "r");
00049     if (!file)
00050     {
00051         return list;  // return an empty list instead of NULL
00052     }
00053
00054     char line[MAX_LINE];
00055
00056     while (fgets(line, sizeof(line), file))
00057     {
00058         line[strcspn(line, "\n")] = 0;
00059         char* title = strdup(line);
00060
00061         if (!fgets(line, sizeof(line), file)) break;
00062         line[strcspn(line, "\n")] = 0;
00063         char* description = strdup(line);
00064
00065         if (!fgets(line, sizeof(line), file)) break;
00066         line[strcspn(line, "\n")] = 0;
00067         char* deadline = strdup(line);
00068
00069         fgets(line, sizeof(line), file); // Skip '---'
00070
00071         Project* p = create_project(title, description, deadline);
00072
00073         char task_filename[MAX_LINE];
00074         create_task_filename(title, task_filename, sizeof(task_filename));
00075         p->task_list = load_tasks_from_file(task_filename);
00076         push_project(&list, p);
00077
00078         free(title);
00079         free(description);
00080         free(deadline);
00081     }
00082
```

```
00083     fclose(file);
00084     return list;
00085 }
00086
00087
00088 void view_project(ProjectList* list)
00089 {
00090
00091     if (!list->head)
00092     {
00093         printf("No saved projects found.\n");
00094         return;
00095     }
00096
00097     printf("\n--- Projects ---\n");
00098     Project* current = list->head;
00099     uint16_t index = 1;
00100     while (current)
00101     {
00102         printf("%u. %s\n", index++, current->title);
00103         current = current->next;
00104     }
00105
00106     uint16_t choice;
00107     printf("\nEnter the project number to see, or 0 to go back: ");
00108     scanf("%hu", &choice);
00109     while (getchar() != '\n');
00110
00111     if (choice == 0)
00112     {
00113         return;
00114     }
00115
00116     current = list->head;
00117     uint16_t i = 1;
00118     while (current && i < choice)
00119     {
00120         current = current->next;
00121         i++;
00122     }
00123
00124     if (current)
00125     {
00126         printf("\nProject: %s\nDescription: %s\nDeadline: %s\n",
00127                current->title, current->description, current->deadline);
00128         // TODO: manage_project(current);
00129     }
00130     else
00131     {
00132         printf("Invalid choice.\n");
00133     }
00134
00135 }
00136
00137
00138 void add_project(ProjectList* list)
00139 {
00140     while(1)
00141     {
00142         char* title = (char*)malloc(100);
00143         char* description = (char*)malloc(256);
00144         char* deadline = (char*)malloc(20);
00145
00146         if (!title || !description || !deadline)
00147         {
00148             printf("Memory allocation failed.\n");
00149             return;
00150         }
00151
00152         printf("Enter project title: ");
00153         scanf(" %[^\n]", title);
00154
00155         printf("Enter project description: ");
00156         scanf(" %[^\n]", description);
00157
00158         printf("Enter project deadline (YYYY-MM-DD): ");
00159         scanf(" %[^\n]", deadline);
00160
00161         Project* new_proj = create_project(title, description, deadline);
00162         printf("\nAdding Project.....\n");
00163         push_project(list, new_proj);
00164
00165         save_project_to_file(new_proj);  // appends one new project to file
00166
00167         printf("Project added successfully.\n");
00168
00169         free(title);
```

```
00170         free(description);
00171         free(deadline);
00172 
00173         char choice;
00174         printf("\nWould you like to add another project? (y/n): ");
00175         scanf(" %c", &choice);
00176         while (getchar() != '\n'); // clear input buffer
00177 
00178         if (choice != 'y' && choice != 'Y')
00179         {
00180             break;
00181         }
00182     }
00183 }
00184 
00185 void save_project_to_file(Project* project)
00186 {
00187     FILE* file = fopen("projectlist.txt", "a"); // append mode
00188     if (!file)
00189     {
00190         printf("Error opening file to save project.\n");
00191         return;
00192     }
00193 
00194     fprintf(file, "%s\n", project->title);
00195     fprintf(file, "%s\n", project->description);
00196     fprintf(file, "%s\n", project->deadline);
00197     fprintf(file, "---\n");
00198 
00199     fclose(file);
00200 }
00201 
00202 void free_project_list(ProjectList *list){
00203     Project* current = list->head;
00204     while (current) {
00205         Project* next = current->next;
00206         free(current->title);
00207         free(current->description);
00208         free(current->deadline);
00209 
00210         if (current->task_list) {
00211             free_task_list(current->task_list);
00212         }
00213 
00214         free(current);
00215         current = next;
00216     }
00217     list->head = NULL;
00218 }
00219 
00220 #include <stdio.h>
00221 #include "project.h"
00222 #include "task.h"
00223 #include "ui.h"
00224 void edit_project(ProjectList* list)
00225 {
00226     if (!list || !list->head) {
00227         printf("No projects to edit.\n");
00228         return;
00229     }
00230 
00231     printf("\n--- Select Project to Edit ---\n");
00232     Project* current = list->head;
00233     unsigned index = 1;
00234     while (current) {
00235         printf("%u. %s\n", index++, current->title);
00236         current = current->next;
00237     }
00238 
00239     printf("Enter project number to edit (or 0 to cancel): ");
00240     unsigned choice;
00241     scanf("%u", &choice);
00242     while (getchar() != '\n');
00243 
00244     if (choice == 0) return;
00245 
00246     current = list->head;
00247     unsigned i = 1;
00248     while (current && i < choice) {
00249         current = current->next;
00250         i++;
00251     }
00252 
00253     if (!current) {
00254         printf("Invalid selection.\n");
00255         return;
00256     }
```

```
00257
00258     char new_title[100], new_description[256], new_deadline[20];
00259     printf("Enter new title (or press Enter to keep '%s'): ", current->title);
00260     fgets(new_title, sizeof(new_title), stdin);
00261     new_title[strcspn(new_title, "\n")] = 0;
00262
00263     printf("Enter new description (or press Enter to keep current): ");
00264     fgets(new_description, sizeof(new_description), stdin);
00265     new_description[strcspn(new_description, "\n")] = 0;
00266
00267     printf("Enter new deadline (YYYY-MM-DD) (or press Enter to keep current): ");
00268     fgets(new_deadline, sizeof(new_deadline), stdin);
00269     new_deadline[strcspn(new_deadline, "\n")] = 0;
00270
00271     // Save old task filename if title changed
00272     char old_task_filename[100];
00273     create_task_filename(current->title, old_task_filename, sizeof(old_task_filename));
00274
00275     if (strlen(new_title) > 0) {
00276         free(current->title);
00277         current->title = strdup(new_title);
00278     }
00279     if (strlen(new_description) > 0) {
00280         free(current->description);
00281         current->description = strdup(new_description);
00282     }
00283     if (strlen(new_deadline) > 0) {
00284         free(current->deadline);
00285         current->deadline = strdup(new_deadline);
00286     }
00287
00288     // If title was changed, rename task file
00289     char new_task_filename[100];
00290     create_task_filename(current->title, new_task_filename, sizeof(new_task_filename));
00291     if (strcmp(old_task_filename, new_task_filename) != 0) {
00292         rename(old_task_filename, new_task_filename);
00293     }
00294
00295     // Rewrite entire project list to file
00296     FILE* file = fopen("projectlist.txt", "w");
00297     if (!file) {
00298         printf("Error updating project file.\n");
00299         return;
00300     }
00301
00302     Project* iter = list->head;
00303     while (iter) {
00304         fprintf(file, "%s\n%s\n%s\n---\n", iter->title, iter->description, iter->deadline);
00305         iter = iter->next;
00306     }
00307     fclose(file);
00308
00309     printf("Project updated successfully.\n");
00310 }
00311 void delete_project(ProjectList* list)
00312 {
00313     if (!list || !list->head) {
00314         printf("No projects to delete.\n");
00315         return;
00316     }
00317
00318     printf("\n--- Select Project to Delete ---\n");
00319     Project* current = list->head;
00320     unsigned index = 1;
00321     while (current) {
00322         printf("%u. %s\n", index++, current->title);
00323         current = current->next;
00324     }
00325
00326     printf("Enter project number to delete (or 0 to cancel): ");
00327     unsigned choice;
00328     scanf("%u", &choice);
00329     while (getchar() != '\n');
00330
00331     if (choice == 0) return;
00332
00333     Project* prev = NULL;
00334     current = list->head;
00335     unsigned i = 1;
00336     while (current && i < choice) {
00337         prev = current;
00338         current = current->next;
00339         i++;
00340     }
00341
00342     if (!current) {
00343         printf("Invalid selection.\n");
```

```
00344         return;
00345     }
00346
00347     // Confirm deletion
00348     printf("Are you sure you want to delete project '%s'? (y/n): ", current->title);
00349     char confirm;
00350     scanf(" %c", &confirm);
00351     while (getchar() != '\n');
00352
00353     if (confirm != 'y' && confirm != 'Y') return;
00354
00355     // Remove from linked list
00356     if (prev) prev->next = current->next;
00357     else list->head = current->next;
00358
00359     // Delete task file
00360     char task_filename[100];
00361     create_task_filename(current->title, task_filename, sizeof(task_filename));
00362     remove(task_filename);
00363
00364     // Free memory
00365     free(current->title);
00366     free(current->description);
00367     free(current->deadline);
00368     if (current->task_list) free_task_list(current->task_list);
00369     free(current);
00370
00371     // Rewrite projectlist.txt
00372     FILE* file = fopen("projectlist.txt", "w");
00373     if (!file) {
00374         printf("Error updating project file.\n");
00375         return;
00376     }
00377     current = list->head;
00378     while (current) {
00379         fprintf(file, "%s\n%s\n%s\n---\n", current->title, current->description, current->deadline);
00380         current = current->next;
00381     }
00382     fclose(file);
00383
00384     printf("Project deleted successfully.\n");
00385 }
00386 void manage_project(ProjectList* list) {
00387     if (!list || !list->head) {
00388         printf("No projects available to manage.\n");
00389         return;
00390     }
00391
00392     // Show list of projects
00393     printf("\n--- Select Project to Manage ---\n");
00394     Project* current = list->head;
00395     unsigned index = 1;
00396     while (current) {
00397         printf("%u. %s\n", index++, current->title);
00398         current = current->next;
00399     }
00400
00401     // User input
00402     printf("Enter project number (or 0 to go back): ");
00403     unsigned choice;
00404     scanf("%u", &choice);
00405     while (getchar() != '\n');
00406
00407     if (choice == 0) return;
00408
00409     // Locate selected project
00410     current = list->head;
00411     unsigned i = 1;
00412     while (current && i < choice) {
00413         current = current->next;
00414         i++;
00415     }
00416
00417     if (!current) {
00418         printf("Invalid selection.\n");
00419         return;
00420     }
00421
00422     // Load tasks for selected project (if not already loaded)
00423     if (!current->task_list) {
00424         current->task_list = (TaskList*)malloc(sizeof(TaskList));
00425         current->task_list->head = NULL;
00426
00427         char task_filename[100];
00428         create_task_filename(current->title, task_filename, sizeof(task_filename));
00429         TaskList* loaded = load_tasks_from_file(task_filename);
00430         if (loaded) {
```

```
00431             free(current->task_list);  // Replace empty task_list with loaded
00432             current->task_list = loaded;
00433         }
00434     }
00435
00436
00437     // Menu loop
00438     int running = 1;
00439     while (running) {
00440         printf("\n--- Managing Project: %s ---\n", current->title);
00441         display_manage_menu();  // UI function you already created
00442
00443         printf("Enter choice: ");
00444         unsigned option;
00445         scanf("%u", &option);
00446         while (getchar() != '\n');
00447
00448         char task_filename[100];
00449         create_task_filename(current->title, task_filename, sizeof(task_filename));
00450
00451         switch (option) {
00452     case 1:
00453         view_tasks(current->task_list);
00454         break;
00455     case 2:
00456         add_task(current->task_list, current->title);
00457         break;
00458     case 3:
00459         edit_task(current->task_list, current->title);  //  use new function
00460         break;
00461     case 4:
00462         delete_task(current->task_list, current->title);  //  use new function
00463         break;
00464     case 5:
00465         printf("Reorder task not implemented yet.\n");
00466         break;
00467     case 6:
00468         printf("Returning back to main menu....\n");
00469         running = 0;
00470         break;
00471     default:
00472         printf("Invalid option. Try again.\n");
00473 }
00474
00475     }
00476 }
```

## 4.9 project.h File Reference

```
#include <stdint.h>
#include "task.h"
```

### Data Structures

- struct Project
- struct ProjectList

### Typedefs

- typedef struct Project Project

**Functions**

- Project ∗ create_project (const char ∗title, const char ∗description, const char ∗deadline)
- void push_project (ProjectList ∗list, Project ∗new_project)
- void add_project (ProjectList ∗list)
- void view_project ()
- ProjectList load_project (const char ∗filename)
- void save_project_to_file (Project ∗project)
- void free_project_list (ProjectList ∗list)
- void manage_project (ProjectList ∗list)
- void edit_project (ProjectList ∗list)
- void delete_project (ProjectList ∗list)

### 4.9.1 Typedef Documentation

#### 4.9.1.1 Project

```
typedef struct Project Project
```

### 4.9.2 Function Documentation

#### 4.9.2.1 add_project()

```
void add_project (
            ProjectList * list)
```

Definition at line 138 of file project.c.

#### 4.9.2.2 create_project()

```
Project * create_project (
            const char * title,
            const char * description,
            const char * deadline)
```

Definition at line 12 of file project.c.

#### 4.9.2.3 delete_project()

```
void delete_project (
            ProjectList * list)
```

Definition at line 311 of file project.c.

#### 4.9.2.4 edit_project()

```
void edit_project (
            ProjectList * list)
```

Definition at line 224 of file project.c.

#### 4.9.2.5 free_project_list()

```
void free_project_list (
            ProjectList * list)
```

Definition at line 202 of file project.c.

#### 4.9.2.6 load_project()

```
ProjectList load_project (
            const char * filename)
```

Definition at line 43 of file project.c.

#### 4.9.2.7 manage_project()

```
void manage_project (
            ProjectList * list)
```

Definition at line 386 of file project.c.

#### 4.9.2.8 push_project()

```
void push_project (
            ProjectList * list,
            Project * new_project)
```

Definition at line 28 of file project.c.

#### 4.9.2.9 save_project_to_file()

```
void save_project_to_file (
            Project * project)
```

Definition at line 185 of file project.c.

#### 4.9.2.10 view_project()

```
void view_project ()
```

## 4.10 project.h

Go to the documentation of this file.

```
00001 #ifndef PROJECT_H
00002 #define PROJECT_H
00003
00004 #include <stdint.h>
00005 #include"task.h"
00006
00007 typedef struct Project {
00008     char* title;
00009     char* description;
00010     char* deadline;
00011     TaskList* task_list;
00012     struct Project* next;
00013 } Project;
00014
00015 typedef struct {
00016     Project* head;
00017 } ProjectList;
00018
00019 // Project management functions
00020 Project* create_project(const char* title, const char* description, const char* deadline);  // Takes
      user input inside
00021 void push_project(ProjectList* list, Project* new_project);
00022 void add_project(ProjectList* list);  // Adds a new project from user input
00023 void view_project();
00024 ProjectList load_project(const char* filename);
00025 void save_project_to_file(Project* project);
00026 void free_project_list(ProjectList *list);
00027 void manage_project(ProjectList* list);
00028 void edit_project(ProjectList* list);
00029 void delete_project(ProjectList* list);
00030
00031
00032 #endif
```

## 4.11 task.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "task.h"
#include "ui.h"
#include "utils.h"
#include "priority.h"
```

### Functions

- Task * create_task (const char *title, const char *description, const char *deadline, Priority priority, TaskStatus status)
- void push_task (TaskList *list, Task *new_task)
- void add_task (TaskList *list, const char *project_title)
- void view_tasks (TaskList *list)
- void save_tasks_to_file (TaskList *task_list, const char *filename)
- TaskList * load_tasks_from_file (const char *filename)
- void edit_task (TaskList *list, const char *project_title)
- void delete_task (TaskList *list, const char *project_title)
- void free_task_list (TaskList *task_list)
- void create_task_filename (const char *project_title, char *buffer, size_t buffer_size)

### 4.11.1 Function Documentation

#### 4.11.1.1 add_task()

```
void add_task (
            TaskList * list,
            const char * project_title)
```

Definition at line 42 of file task.c.

#### 4.11.1.2 create_task()

```
Task * create_task (
            const char * title,
            const char * description,
            const char * deadline,
            Priority priority,
            TaskStatus status)
```

Definition at line 12 of file task.c.

#### 4.11.1.3 create_task_filename()

```
void create_task_filename (
            const char * project_title,
            char * buffer,
            size_t buffer_size)
```

Definition at line 362 of file task.c.

#### 4.11.1.4 delete_task()

```
void delete_task (
            TaskList * list,
            const char * project_title)
```

Definition at line 282 of file task.c.

#### 4.11.1.5 edit_task()

```
void edit_task (
            TaskList * list,
            const char * project_title)
```

Definition at line 193 of file task.c.

#### 4.11.1.6 free_task_list()

```
void free_task_list (
            TaskList * task_list)
```

Definition at line 346 of file task.c.

#### 4.11.1.7 load_tasks_from_file()

```
TaskList * load_tasks_from_file (
            const char * filename)
```

Definition at line 146 of file task.c.

#### 4.11.1.8 push_task()

```
void push_task (
            TaskList * list,
            Task * new_task)
```

Definition at line 27 of file task.c.

#### 4.11.1.9 save_tasks_to_file()

```
void save_tasks_to_file (
            TaskList * task_list,
            const char * filename)
```

Definition at line 121 of file task.c.

#### 4.11.1.10 view_tasks()

```
void view_tasks (
            TaskList * list)
```

Definition at line 99 of file task.c.

# 4.12 task.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <string.h>
00004 #include <stdint.h>
00005 #include "task.h"
00006 #include "ui.h"
00007 #include "utils.h"
00008 #include "priority.h"
00009
00010
00011
00012 Task* create_task(const char* title, const char* description, const char* deadline, Priority priority,
      TaskStatus status)
00013 {
00014     Task* t = (Task*)malloc(sizeof(Task));
00015     if (!t) return NULL;
00016
00017     t->title = strdup(title);
00018     t->description = strdup(description);
00019     t->deadline = strdup(deadline);
00020     t->priority_level = priority;
00021     t->status = status;
00022     t->next = NULL;
00023
00024     return t;
00025 }
00026
00027 void push_task(TaskList* list, Task* new_task)
00028 {
00029     if (!list->head)
00030     {
00031         list->head = new_task;
00032         return;
00033     }
00034
00035     Task* current = list->head;
00036     while (current->next)
00037         current = current->next;
00038
00039     current->next = new_task;
00040 }
00041
00042 void add_task(TaskList* list,const char* project_title)
00043 {
00044     while (1)
00045     {
00046         char* title = (char*)malloc(100);
00047         char* description = (char*)malloc(256);
00048         char* deadline = (char*)malloc(20);
00049         unsigned priority_input, status_input;
00050
00051         if (!title || !description || !deadline)
00052         {
00053             printf("Memory allocation failed.\n");
00054             return;
00055         }
00056
00057         printf("Enter task title: ");
00058         scanf(" %[^\n]", title);
00059         while (getchar() != '\n');
00060
00061         printf("Enter task description: ");
00062         scanf(" %[^\n]", description);
00063         while (getchar() != '\n');
00064
00065         printf("Enter task deadline (YYYY-MM-DD): ");
00066         scanf(" %[^\n]", deadline);
00067         while (getchar() != '\n');
00068
00069         printf("Enter priority (1 = Low, 2 = Medium, 3 = High): ");
00070         scanf("%u", &priority_input);
00071         while (getchar() != '\n');
00072
00073         printf("Enter status (0 = TODO, 1 = IN_PROGRESS, 2 = DONE): ");
00074         scanf("%u", &status_input);
00075         while (getchar() != '\n');
00076
00077         Task* new_task = create_task(title, description, deadline, (Priority)priority_input,
      (TaskStatus)status_input);
00078         push_task(list, new_task);
00079
00080         char filename[100];
```

```
00081         create_task_filename(project_title, filename, sizeof(filename));
00082         save_tasks_to_file(list, filename);
00083
00084         printf("Task added successfully.\n");
00085
00086         free(title);
00087         free(description);
00088         free(deadline);
00089
00090         char choice;
00091         printf("\nWould you like to add another task? (y/n): ");
00092         scanf(" %c", &choice);
00093         while (getchar() != '\n');
00094
00095         if (choice != 'y' && choice != 'Y') break;
00096     }
00097 }
00098
00099 void view_tasks(TaskList* list)
00100 {
00101     if (!list->head)
00102     {
00103         printf("No tasks available.\n");
00104         return;
00105     }
00106
00107     Task* current = list->head;
00108     unsigned index = 1;
00109     while (current)
00110     {
00111         printf("\nTask %u:\n", index++);
00112         printf("Title       : %s\n", current->title);
00113         printf("Description: %s\n", current->description);
00114         printf("Deadline    : %s\n", current->deadline);
00115         printf("Priority    : %u\n", current->priority_level);
00116         printf("Status      : %u\n", current->status);
00117         current = current->next;
00118     }
00119 }
00120
00121 void save_tasks_to_file(TaskList* task_list, const char* filename)
00122 {
00123     FILE* file = fopen(filename, "a"); // append mode
00124     if (!file)
00125     {
00126         printf("Error opening task file.\n");
00127         return;
00128     }
00129
00130     Task* current = task_list->head;
00131     while (current)
00132     {
00133         fprintf(file, "%s\n", current->title);
00134         fprintf(file, "%s\n", current->description);
00135         fprintf(file, "%s\n", current->deadline);
00136         fprintf(file, "%u\n", (unsigned)current->priority_level);
00137         fprintf(file, "%u\n", (unsigned)current->status);
00138         fprintf(file, "---\n");
00139
00140         current = current->next;
00141     }
00142
00143     fclose(file);
00144 }
00145
00146 TaskList* load_tasks_from_file(const char* filename)
00147 {
00148     FILE* file = fopen(filename, "r");
00149     TaskList* list = (TaskList*)malloc(sizeof(TaskList));
00150     if (!list) return NULL;
00151     list->head = NULL;
00152
00153     if (!file)
00154     {
00155         // If the file doesn't exist, return an empty task list
00156         return list;
00157     }
00158
00159     char line[256];
00160     while (fgets(line, sizeof(line), file))
00161     {
00162         line[strcspn(line, "\n")] = 0;
00163         char* title = strdup(line);
00164
00165         if (!fgets(line, sizeof(line), file)) break;
00166         line[strcspn(line, "\n")] = 0;
00167         char* description = strdup(line);
```

```
00168
00169         if (!fgets(line, sizeof(line), file)) break;
00170         line[strcspn(line, "\n")] = 0;
00171         char* deadline = strdup(line);
00172
00173         if (!fgets(line, sizeof(line), file)) break;
00174         unsigned int priority = (unsigned int)atoi(line);
00175
00176         if (!fgets(line, sizeof(line), file)) break;
00177         unsigned int status = (unsigned int)atoi(line);
00178
00179         fgets(line, sizeof(line), file); // Skip the "---" separator
00180
00181         Task* task = create_task(title, description, deadline, priority, status);
00182         push_task(list, task);
00183
00184         free(title);
00185         free(description);
00186         free(deadline);
00187     }
00188
00189     fclose(file);
00190     return list;
00191 }
00192 //Edit task
00193 void edit_task(TaskList* list, const char* project_title)
00194 {
00195     if (!list || !list->head)
00196     {
00197         printf("No tasks to edit.\n");
00198         return;
00199     }
00200
00201     view_tasks(list);
00202
00203     printf("Enter the task number to edit: ");
00204     int task_num;
00205     scanf("%d", &task_num);
00206     while (getchar() != '\n');
00207
00208     Task* current = list->head;
00209     int i = 1;
00210     while (current && i < task_num)
00211     {
00212         current = current->next;
00213         i++;
00214     }
00215
00216     if (!current)
00217     {
00218         printf("Invalid task number.\n");
00219         return;
00220     }
00221
00222     char buffer[100];
00223
00224     printf("Enter new title (or press Enter to keep '%s'): ", current->title);
00225     fgets(buffer, sizeof(buffer), stdin);
00226     buffer[strcspn(buffer, "\n")] = 0;
00227     if (strlen(buffer) > 0)
00228     {
00229         free(current->title);
00230         current->title = strdup(buffer);
00231     }
00232
00233     printf("Enter new description (or press Enter to keep current): ");
00234     fgets(buffer, sizeof(buffer), stdin);
00235     buffer[strcspn(buffer, "\n")] = 0;
00236     if (strlen(buffer) > 0)
00237     {
00238         free(current->description);
00239         current->description = strdup(buffer);
00240     }
00241
00242     printf("Enter new deadline (YYYY-MM-DD) (or press Enter to keep current): ");
00243     fgets(buffer, sizeof(buffer), stdin);
00244     buffer[strcspn(buffer, "\n")] = 0;
00245     if (strlen(buffer) > 0)
00246     {
00247         free(current->deadline);
00248         current->deadline = strdup(buffer);
00249     }
00250
00251     unsigned int new_priority;
00252     printf("Enter new priority (1=Low, 2=Medium, 3=High), or 0 to keep current: ");
00253     scanf("%u", &new_priority);
00254     while (getchar() != '\n');
```

```
00255     if (new_priority >= 1 && new_priority <= 3)
00256         current->priority_level = (Priority)new_priority;
00257
00258     unsigned int new_status;
00259     printf("Enter new status (0=TODO, 1=IN_PROGRESS, 2=DONE), or 9 to skip: ");
00260     scanf("%u", &new_status);
00261     while (getchar() != '\n');
00262     if (new_status <= 2)
00263         current->status = (TaskStatus)new_status;
00264
00265     char filename[100];
00266     create_task_filename(project_title, filename, sizeof(filename));
00267     FILE* file = fopen(filename, "w");
00268     Task* temp = list->head;
00269     while (temp)
00270     {
00271         fprintf(file, "%s\n%s\n%s\n%u\n%u\n---\n",
00272                 temp->title, temp->description, temp->deadline,
00273                 (unsigned)temp->priority_level, (unsigned)temp->status);
00274         temp = temp->next;
00275     }
00276     fclose(file);
00277
00278     printf("Task updated successfully.\n");
00279 }
00280
00281 //Delete task
00282 void delete_task(TaskList* list, const char* project_title)
00283 {
00284     if (!list || !list->head)
00285     {
00286         printf("No tasks to delete.\n");
00287         return;
00288     }
00289
00290     view_tasks(list);
00291
00292     printf("Enter the task number to delete: ");
00293     int task_num;
00294     scanf("%d", &task_num);
00295     while (getchar() != '\n');
00296
00297     Task* current = list->head;
00298     Task* prev = NULL;
00299     int i = 1;
00300     while (current && i < task_num)
00301     {
00302         prev = current;
00303         current = current->next;
00304         i++;
00305     }
00306
00307     if (!current)
00308     {
00309         printf("Invalid task number.\n");
00310         return;
00311     }
00312
00313     printf("Are you sure you want to delete '%s'? (y/n): ", current->title);
00314     char confirm;
00315     scanf(" %c", &confirm);
00316     while (getchar() != '\n');
00317     if (confirm != 'y' && confirm != 'Y') return;
00318
00319     if (prev)
00320         prev->next = current->next;
00321     else
00322         list->head = current->next;
00323
00324     free(current->title);
00325     free(current->description);
00326     free(current->deadline);
00327     free(current);
00328
00329     // Save updated task list
00330     char filename[100];
00331     create_task_filename(project_title, filename, sizeof(filename));
00332     FILE* file = fopen(filename, "w");
00333     Task* temp = list->head;
00334     while (temp)
00335     {
00336         fprintf(file, "%s\n%s\n%s\n%u\n%u\n---\n",
00337                 temp->title, temp->description, temp->deadline,
00338                 (unsigned)temp->priority_level, (unsigned)temp->status);
00339         temp = temp->next;
00340     }
00341     fclose(file);
```

```
00342
00343     printf("Task deleted successfully.\n");
00344 }
00345
00346 void free_task_list(TaskList* task_list)
00347 {
00348     Task* current = task_list->head;
00349     while (current)
00350     {
00351         Task* next = current->next;
00352         free(current->title);
00353         free(current->description);
00354         free(current->deadline);
00355         free(current);
00356         current = next;
00357     }
00358     task_list->head = NULL;
00359     free(task_list);
00360 }
00361
00362 void create_task_filename(const char* project_title, char* buffer, size_t buffer_size)
00363 {
00364     snprintf(buffer, buffer_size, "tasks_%s.txt", project_title);
00365 }
00366
```

## 4.13 task.h File Reference

```
#include "utils.h"
#include "priority.h"
```

### Data Structures

- struct Task
- struct TaskList

### Typedefs

- typedef struct Task Task

### Enumerations

- enum TaskStatus { TODO , IN_PROGRESS , DONE }

### Functions

- Task * create_task (const char *title, const char *description, const char *deadline, Priority priority, TaskStatus status)
- void push_task (TaskList *task_list, Task *new_task)
- void add_task (TaskList *task_list, const char *project_title)
- void view_tasks_by_status (TaskList *task_list, TaskStatus status)
- void save_tasks_to_file (TaskList *task_list, const char *filename)
- TaskList * load_tasks_from_file (const char *filename)
- void update_task_status (TaskList *task_list, int task_number, TaskStatus new_status)
- void free_task_list (TaskList *task_list)
- void create_task_filename (const char *project_title, char *buffer, size_t buffer_size)
- void edit_task (TaskList *list, const char *project_title)
- void delete_task (TaskList *list, const char *project_title)

### 4.13.1 Typedef Documentation

#### 4.13.1.1 Task

```
typedef struct Task Task
```

### 4.13.2 Enumeration Type Documentation

#### 4.13.2.1 TaskStatus

```
enum TaskStatus
```

**Enumerator**

| | |
|---|---|
| TODO | |
| IN_PROGRESS | |
| DONE | |

Definition at line 6 of file task.h.

### 4.13.3 Function Documentation

#### 4.13.3.1 add_task()

```
void add_task (
            TaskList * task_list,
            const char * project_title)
```

Definition at line 42 of file task.c.

#### 4.13.3.2 create_task()

```
Task * create_task (
            const char * title,
            const char * description,
            const char * deadline,
            Priority priority,
            TaskStatus status)
```

Definition at line 12 of file task.c.

#### 4.13.3.3 create_task_filename()

```
void create_task_filename (
            const char * project_title,
            char * buffer,
            size_t buffer_size)
```

Definition at line 362 of file task.c.

#### 4.13.3.4 delete_task()

```
void delete_task (
            TaskList * list,
            const char * project_title)
```

Definition at line 282 of file task.c.

#### 4.13.3.5 edit_task()

```
void edit_task (
            TaskList * list,
            const char * project_title)
```

Definition at line 193 of file task.c.

#### 4.13.3.6 free_task_list()

```
void free_task_list (
            TaskList * task_list)
```

Definition at line 346 of file task.c.

#### 4.13.3.7 load_tasks_from_file()

```
TaskList * load_tasks_from_file (
            const char * filename)
```

Definition at line 146 of file task.c.

#### 4.13.3.8 push_task()

```
void push_task (
            TaskList * task_list,
            Task * new_task)
```

Definition at line 27 of file task.c.

#### 4.13.3.9 save_tasks_to_file()

```
void save_tasks_to_file (
            TaskList * task_list,
            const char * filename)
```

Definition at line 121 of file task.c.

#### 4.13.3.10 update_task_status()

```
void update_task_status (
            TaskList * task_list,
            int task_number,
            TaskStatus new_status)
```

#### 4.13.3.11 view_tasks_by_status()

```
void view_tasks_by_status (
            TaskList * task_list,
            TaskStatus status)
```

## 4.14 task.h

Go to the documentation of this file.

```
00001 #ifndef TASK_H
00002 #define TASK_H
00003 #include "utils.h"
00004 #include "priority.h"
00005
00006 typedef enum
00007 {
00008     TODO,
00009     IN_PROGRESS,
00010     DONE
00011 } TaskStatus;
00012
00013 typedef struct Task
00014 {
00015     char* title;
00016     char* description;
00017     char* deadline;
00018     Priority priority_level;
00019     TaskStatus status;
00020     struct Task* next;
00021 } Task;
00022
00023 typedef struct
00024 {
00025     Task* head;
00026 } TaskList;
00027 // Task management functions
00028 Task* create_task(const char* title, const char* description, const char* deadline, Priority priority,
      TaskStatus status);
00029 void push_task(TaskList* task_list, Task* new_task);
00030 void add_task(TaskList* task_list,const char* project_title);
00031 void view_tasks_by_status(TaskList* task_list, TaskStatus status);
00032 void save_tasks_to_file(TaskList* task_list, const char* filename);
00033 TaskList* load_tasks_from_file(const char* filename);
00034 void update_task_status(TaskList* task_list, int task_number, TaskStatus new_status);
00035 void free_task_list(TaskList* task_list);
00036 void create_task_filename(const char* project_title, char* buffer, size_t buffer_size);
00037 void edit_task(TaskList* list, const char* project_title);
00038 void delete_task(TaskList* list, const char* project_title);
00039 #endif
```

## 4.15 ui.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include "ui.h"
#include "project.h"
#include <windows.h>
```

### Functions

- void loading_animation ()
- void start_interface ()
- void display_main_menu ()
- void display_manage_menu ()
- void display_task_menu ()
- void clear_screen ()
- void pause_and_clear ()

## 4.15.1 Function Documentation

### 4.15.1.1 clear_screen()

```
void clear_screen ()
```

Definition at line 92 of file ui.c.

### 4.15.1.2 display_main_menu()

```
void display_main_menu ()
```

Definition at line 61 of file ui.c.

### 4.15.1.3 display_manage_menu()

```
void display_manage_menu ()
```

Definition at line 71 of file ui.c.

### 4.15.1.4 display_task_menu()

```
void display_task_menu ()
```

Definition at line 83 of file ui.c.

### 4.15.1.5 loading_animation()

```
void loading_animation ()
```

Definition at line 9 of file ui.c.

### 4.15.1.6 pause_and_clear()

```
void pause_and_clear ()
```

Definition at line 100 of file ui.c.

#### 4.15.1.7 start_interface()

```
void start_interface ()
```

Definition at line 19 of file ui.c.

## 4.16 ui.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include "ui.h"
00004 #include "project.h"
00005 //#define BLUE    "\x1b[34m"
00006 //#define GREEN   "\x1b[32m"
00007 //#define RESET   "\x1b[0m"
00008 #include<windows.h>
00009 void loading_animation() {
00010     printf("Loading project");
00011     for (int i = 0; i < 3; i++) {
00012         printf(".");
00013         fflush(stdout);
00014         Sleep(1000);  // On Linux/macOS
00015     }
00016     printf("\n");
00017 }
00018
00019 void start_interface() {
00020
00021     ProjectList list = load_project("projectlist.txt");
00022     while (1) {
00023         clear_screen();
00024         display_main_menu();
00025
00026         int choice;
00027         printf("Enter your choice: ");
00028         scanf("%d", &choice);
00029         while (getchar() != '\n'); // Clear input buffer
00030
00031         switch (choice) {
00032     case 1:
00033         view_project(&list);
00034         break;
00035     case 2:
00036         add_project(&list);
00037         break;
00038     case 3:
00039         manage_project(&list);
00040         break;
00041     case 4:
00042         edit_project(&list);
00043         break;
00044     case 5:
00045         delete_project(&list);
00046         break;
00047     case 6:
00048         free_project_list(&list);
00049         printf("Exiting. Thank you for using Trello\n");
00050         return;
00051     default:
00052         printf("Invalid choice. Please try again.\n");
00053 }
00054
00055
00056         pause_and_clear();
00057     }
00058 }
00059
00060
00061 void display_main_menu() {
00062     printf("\x1b[97m""\n=====  Trello in C =====\n""\x1b[0m");
00063     printf("\x1b[96m""1. View All Projects\n");
00064     printf("2. Create a New Project\n");
00065     printf("3. Manage Project\n");
00066     printf("4. Edit Project\n");
00067     printf("5. Delete Project\n");
00068     printf("6. Exit\n");
00069 }
```

```
00070
00071 void display_manage_menu(){
00072     printf("\n--- Manage Project: project name ---\n");
00073     printf("1. View Tasks by Status\n");
00074     printf("2. Add New Task\n");
00075     printf("3. Edit Task\n");
00076     printf("4. Delete Task\n");
00077     printf("5. Reorder\n");
00078     printf("6. Go back\n");
00079
00080 }
00081
00082
00083 void display_task_menu() {
00084     printf("\n--- project name ---\n");
00085     printf("1. View Lists\n");
00086     printf("2. Edit Card (Task)\n");
00087     printf("3. Delete Card (Task)\n");
00088     printf("4. Reorder Cards (Tasks)\n");
00089     printf("5. View Cards (Tasks)\n");
00090     printf("6. Back\n");
00091 }
00092 void clear_screen() {
00093 #ifdef _WIN32
00094     system("cls");
00095 #else
00096     system("clear");
00097 #endif
00098 }
00099
00100 void pause_and_clear() {
00101     printf("Press Enter to continue...");
00102     getchar();
00103     clear_screen();
00104 }
00105
```

## 4.17 ui.h File Reference

### Functions

- void start_interface ()
- void display_main_menu ()
- void display_manage_menu ()
- void display_task_menu ()
- void loading_animation ()
- void clear_screen ()
- void pause_and_clear ()

### 4.17.1 Function Documentation

#### 4.17.1.1 clear_screen()

```
void clear_screen ()
```

Definition at line 92 of file ui.c.

#### 4.17.1.2 display_main_menu()

```
void display_main_menu ()
```

Definition at line 61 of file ui.c.

#### 4.17.1.3 display_manage_menu()

```
void display_manage_menu ()
```

Definition at line 71 of file ui.c.

#### 4.17.1.4 display_task_menu()

```
void display_task_menu ()
```

Definition at line 83 of file ui.c.

#### 4.17.1.5 loading_animation()

```
void loading_animation ()
```

Definition at line 9 of file ui.c.

#### 4.17.1.6 pause_and_clear()

```
void pause_and_clear ()
```

Definition at line 100 of file ui.c.

#### 4.17.1.7 start_interface()

```
void start_interface ()
```

Definition at line 19 of file ui.c.

## 4.18 ui.h

Go to the documentation of this file.

```
00001 #ifndef UI_H
00002 #define UI_H
00003
00004
00005 void start_interface();
00006 void display_main_menu();
00007 void display_manage_menu();
00008 void display_task_menu();
00009 void loading_animation();
00010 void clear_screen();
00011 void pause_and_clear();
00012
00013 #endif
```

## 4.19 utils.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "utils.h"
#include "priority.h"
```

### Functions

- char * get_string_input (const char *prompt)
- uint16_t get_integer_input (const char *prompt)

### 4.19.1 Function Documentation

#### 4.19.1.1 get_integer_input()

```
uint16_t get_integer_input (
            const char * prompt)
```

Definition at line 30 of file utils.c.

#### 4.19.1.2 get_string_input()

```
char * get_string_input (
            const char * prompt)
```

Definition at line 9 of file utils.c.

## 4.20 utils.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <string.h>
00004 #include <stdint.h>
00005 #include "utils.h"
00006 #include "priority.h"
00007
00008
00009 char* get_string_input(const char* prompt) {
00010     printf("%s", prompt);
00011     char buffer[1024];
00012     fgets(buffer, sizeof(buffer), stdin);
00013
00014     // Remove newline
00015     size_t len = strlen(buffer);
00016     if (len > 0 && buffer[len - 1] == '\n') {
00017         buffer[len - 1] = '\0';
00018     }
00019
00020     // Allocate memory
00021     char* input = (char*)malloc(strlen(buffer) + 1);
00022     if (!input) {
00023         printf("Memory allocation failed.\n");
```

```
00024         exit(1);
00025     }
00026     strcpy(input, buffer);
00027     return input;
00028 }
00029
00030 uint16_t get_integer_input(const char* prompt) {
00031     printf("%s", prompt);
00032     int value;
00033     scanf("%hu", &value);
00034     while (getchar() != '\n'); // Clear input buffer
00035     return value;
00036 }
```

## 4.21 utils.h File Reference

```
#include <stdint.h>
#include "priority.h"
```

### Functions

- char * get_string_input (const char *prompt)
- uint16_t get_integer_input (const char *prompt)

### 4.21.1 Function Documentation

#### 4.21.1.1 get_integer_input()

```
uint16_t get_integer_input (
            const char * prompt)
```

Definition at line 30 of file utils.c.

#### 4.21.1.2 get_string_input()

```
char * get_string_input (
            const char * prompt)
```

Definition at line 9 of file utils.c.

## 4.22 utils.h

Go to the documentation of this file.

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003 #include <stdint.h>
00004 #include "priority.h"
00005 char* get_string_input(const char* prompt);
00006 uint16_t get_integer_input(const char* prompt);
00007
00008 #endif // UTILS_H
```

# Index